Report shell project Operating Systems Concepts

Names & and student numbers: Levente Bódi, s1122062

Your report should be in .pdf, and it should contain your names and student numbers.

In addition, the following sections are required.

1. Implementation

Describe your implementation. Mention how the code traverses the Expression data structure, clearly indicating when pipe(), fork(), and waitpid() are called (8pt).

The *parse_command_line()* function separates the commands, if more given, as an already implemented function. My implementation checks if there are 0, 1, or more commands given. If the command is empty, it terminates the shell process. Besides that, it uses the *chdir(dir)* if the *cd dir* command is given. They run in the parent so that they affect the shell itself. My code does a left-to-right traversal on the commands. For each i from 0 to N-1 (where N is the number of commands, so for all commands except the last one) it creates a pipe for the next command (*pipe(pipefd)*). *Fork()* happens once per command.

When the *pid* is 0, we know we are in the child branch, and for that I created a *STDIN* and *STDOUT* setup. For *STDIN* if it is the first command, it will read from a file input by the user (e.g. *< file.txt*) instead of the keyboard. Otherwise, the child connects the read end of the previous pipe, and this way it receives output of the earlier command. Otherwise, the child connects */dev/null* to its input. If the *STDOUT* is the last command and the user wrote something like *> file.txt* the child opens it for writing so that the results of the command go in that file. Otherwise, it connects the write end of the new pipe to its output. The child also closes all unused file descriptors after setting up redirections.

In the parent branch the parent remembers the child's *pid* and closes the file descriptors it does not need anymore. It closes the *prev_read* if it has been before because the parent is not supposed to read from the pipeline and if a new pipe was created it keeps the read end to pass it along the next child.

After all children are forked, it depends on what happens next whether the job is foreground or background. If it is foreground, the parent waits for the child to finish, using *waitpid()*, however, if it is a background one, the parent does not wait for the children.

To make the shell more robust and reliable, different checks are implemented, according to the description of the assignment as well. To name one, if the user enters more than one argument for *cd* it throws an error that "expected exactly one argument", and in case *cd* fails, it throws a *perror*. These checks are also implemented for the different functionalities described by the assignment.

Provide a list of system calls used by your solution and why/how your program uses them (12pt).

- Pipe() -> creates unidirectional channels between stdout of command i and stdin of command i+1, on failure the pipeline cannot be build further
- Fork() -> creates a new process for each command, on failure, the shell reports the error and skips execution

- Dup2() -> maps output file or current pipe's write end to STDOUT_FILENO or maps input file or previous pipe's read end to STDIN_FILENO, on failure shell reports the error and the command cannot proceed
- Open() -> opens redirection targets (input from and output to files), on failure the child reports the error and terminates
- Close() -> closes all resources (descriptors) to ensure proper EOF behaviour, on failure the shell reports the error, but usually continues since it is not fatal
- Execvp() -> replaces the child process with the desired command, only returns on error
- Waitpid() -> parent waits for the child, on failure it reports the error
- Chdir() -> changes the shell's working directory, on failure the shell reports the error and stays in current directory

The use of these system calls are abundantly defined above.

2. Tests

We expect you to test your shell. Unit tests or a testing framework is not required, and it is sufficient if you just try some commands by hand. Mention the commands you tested. One way to do that, is by making a table with the following columns (10pt, 1pt per test, should have proper coverage):

command	expected result	actual result	Y/N
Built in's			
pwd	show current dir path	/home/levi/my_shell	✅
cd ..	go to parent dir	/home/levi$	✅
cd levi ss	error	cd: expected exactly one argument	✅
cd /no/such/dir	error	cd: No such file or directory	✅
exit	terminate	it terminated	✅
Single commands			
date	output current date	Sun Sep 21 20:02:57 CEST 2025	✅
echo a b c d	output a b c d	a b c d	✅
Pipelines			
date | tail -c 5	last 5 bytes of output	2025 (because it is 2025\n)	✅
echo -e "a\nb\nc" | wc -l	printing 3	3	✅
ls /nope | wc -l	ls -> error, wc -l ->0	ls: cannot access '/nope': No such file or directory 0	✅
Redirections			
echo hello > out.txt	creates/ truncates out.txt with "hello\n"	it did the file with the output correctly	✅
cat < out.txt | wc -c	printing 6 (5 + newline)	6	✅
echo foo | grep x > out.txt	creates/truncates out.txt empty	it was indeed empty	✅
Background			
cat &	will not wait for input	it did not wait as expected	✅
cat | sleep 2 | wc -c &	prompt returns immediately; ~2s later, a lone 0 appears	shell indeed returned immediately and printed 0 after ~2s	✅
cat | sleep 2 | wc -c	prompt waits ~2s and then returns 0	it waited 2s and returned 0	✅
cat < missing.txt | wc -c &	returns file missing error immediately and then prints 0	missing.txt: No such file or directory 0	
Testing exit			
printf "exit\n" | ./build/shell	exits and returns newly built shell	"exit: No such file or directory	❌
printf exit\n | ./build/shell	exits and returns newly built shell	it succeeded – the problem was with user input with trailing "-marks	✅
Some more error testing – command not found, non exec			

nosuchcmd	error on stderr	nosuchcmd: No such file or directory	✅
nosuchcmd: No such file or directory 0	error from nosuchcmd on stderr; wc -l prints 0	nosuchcmd: No such file or directory 0	✅
echo hi | nosuchcmd	nosuchcmd: no such file or directory (stderr), no normal output	nosuchcmd: No such file or directory and no output	✅
mkdir tmpd ./tmpd	permission denial since it is not a binary file but a dir	./tmpd: Permission denied (with cd tmpd it works as it should)	✅
printf foo > noexec ./noexec	same as for tmpd	./noexec: Permission denied	✅
Some more error testing – input (<) errors			
cat < missing.txt	no such file	missing.txt: No such file or directory	✅
cat < tmpd	directory err	cat: -: Is a directory	✅
Some more error testing – output (>) errors			
echo hi > tmpd	dir err	tmpd: Is a directory	✅
mkdir something chmod a-w something echo hi > something/file	permission denial, since we revoked the rights in relation to file 'something'	something/file: Permission denied	✅
printf '%s\n' nosuchcmd exit | ./build/shell 1>out.txt 2>err.txt cat out.txt cat err.txt	expect empty and then no such file/ dir	empty + nosuchcmd: No such file or directory	✅

Given your tests, are you sure your shell does function correctly? Make a distinction between: errors (problems in human), faults (problems in code), failure (problems in execution). (5pt)

I am confident after testing it that it works as expected. Based on the implementation and the manual test matrix it functions properly. Besides, the matrix is quite comprehensive and tests a lot of aspects.

A human made mistake is for example when the user itself makes a mistake. From the matrix it seems that I wrote *printf "exit\n" | ./build/shell* instead of *printf exit\n | ./build/shell* and thus the shell throw an error because of my incorrect syntax, and not because it would have an implementation defect.

A fault in the program is when the shell does not implement job control and reap children in the background leaving them as zombies. It is acceptable but that is an omission in our code relative to a full shell.

A failure would be if we observed our code during runtime to run into deadlocks, wrong redirections, etc. but it did not happen. The error messages we observed are failures of the invoked programs or system calls and not the failures of the shell implementation itself.

Given _any_ tests, can you be sure your shell does function correctly, and handles all edge cases correctly (such as out of memory)? (5pt)

To answer this question shortly, no. Tests are good for abundant and overarching view where our system does well and where could be some weak points or fixes needed. It is like the induction problem, where we can see 1 million swans that are white, but we cannot prove with this that the next one will also be white, because it can be black as well, we are only able to reinforce our belief and see that until so far, the shell coped well with the commands given. Tests are not definitive, and coverage of all extreme cases were not included. It can show that the scenarios were exercised but it cannot guarantee correctness for all cases.

3. Infinite buffer problem

Explain the infinite buffer problem in one paragraph (5pt)

The infinite buffer problem appears when the shell tries to store all the data flows between two commands instead letting the pipe of the kernel move it directly. Because a producer like *cmd1* can generate an arbitrary amount of output, any intermediary that buffered all memory before passing it on would need unbounded memory. With real pipes it is avoidable.

Does your implementation potentially need an infinite buffer for the communication channels between chained commands? Explain why (not), and why your chained commands will terminate properly (if all the subcommands execute correctly). Does the place of waitpid() have an influence on this? (5pt)

In my implementation, no infinite buffer is needed, because the pipelines with *pipe()* + *fork()* + *dup2()* were built, so each command reads and writes the pipe endpoints directly and the shell never slurps command output into RAM. So, the producer writes to a fixed-size buffer (and when it is full the writer blocks until the reader drains it) instead of an unbounded one avoiding the infinite buffer problem. Pipelines terminate properly because every child has exactly one read or write end after dup2, and all unused pipe ends are closed in both children and the parent.

The placement of *waitpid()* is crucial since if it is placed too early it may block the parent while pipes are still open or cause a delayed EOF because the pipe is still open.